SP

SecureProfit Hub

Technical Documentation

Architecture, data model, API & operations

Generated April 18, 2026

This document covers the architecture, project layout, data model, API surface, security model and operational concerns for engineers maintaining SecureProfit Hub.

---

1. Stack Overview

Layer	Technology
Frontend	React 18, Vite, TypeScript, Tailwind CSS, shadcn/ui (Radix), TanStack Query, Wouter (router), Recharts
Backend	Node.js (ESM), Express 5, Prisma 5 ORM, JSON Web Token auth, bcryptjs, multer (uploads), pino logger, zod validation
Database	PostgreSQL (Replit-managed; `DATABASE_URL`)
Build	pnpm workspaces (monorepo), esbuild (server bundle), Vite (client bundle), TypeScript project references
Hosting	Replit Autoscale deployment with mTLS reverse proxy

---

2. Repository Layout

```
.
% % %  artifacts/
%    % % %  api-server/          # Express API
%    %    % % %  src/
%    %    %    % % %  index.ts            # entrypoint, mounts /api router
%    %    %    % % %  lib/                # serializers, audit helper, projectType
%    %    %    % % %  middlewares/auth.ts # requireAuth + requireRole
%    %    %    % % %  routes/             # 1 file per resource
%    %    % % %  build.mjs              # esbuild bundler
%    % % %  web/                 # React + Vite SPA
%    %    % % %  src/
%    %    %    % % %  App.tsx             # router + providers
%    %    %    % % %  pages/              # 1 folder per route
%    %    %    % % %  components/         # ui/ (shadcn), layout/, etc.
%    %    %    % % %  lib/                # auth, format, projectType
%    %    % % %  vite.config.ts
%    % % %  mockup-sandbox/      # Component preview workspace (dev only)
% % %  lib/
%    % % %  db/                  # Prisma schema, generated client, seed
%    %    % % %  prisma/schema.prisma
%    % % %  api-spec/            # OpenAPI source of truth
%    % % %  api-client-react/    # Orval-generated React Query hooks
%    % % %  api-zod/            # Zod schemas shared with the server
% % %  docs/                  # !• you are here
% % %  pnpm-workspace.yaml
```

The repo is a pnpm monorepo. Each artifact in `artifacts/` is registered with the Replit workspace and mapped to its own preview path.

---

3. Data Model (Prisma)

Defined in `lib/db/prisma/schema.prisma`. Summary of models:

`User`

`id, email (unique), passwordHash, name, role, title, dailyRate, isActive, deletedAt, createdAt, updatedAt`

- `role: UserRole` enum: `MANAGEMENT, PROJECT_MANAGER, SALES, KONSULTAN, TECHNICAL_WRITER, ADMIN_PROJECT`
- **Soft delete**: `deletedAt` is set instead of hard delete; `isActive` is also set to `false`. Login middleware rejects deleted users.

`Client`

`id, name, contactPerson, email, phone, industry, createdAt, updatedAt`

`Project`

`id, code (unique SPK/PO), name, description, status, clientId, salesId, pmId, startDate, endDate, contractValue, estimatedCost, plannedMandays, lastStatusReason, deletedAt, createdAt, updatedAt`

- `status: ProjectStatus` enum: `OBSERVATION, ACTIVE, PAUSE, COMPLETE, CLOSED`
- **Soft delete** via `deletedAt`. All list queries filter `deletedAt: null` by default.

`ProjectResource`

Join table: `projectId, userId, roleInProject, plannedMandays, dailyRate`. Unique on `(projectId, userId)`.

`Timesheet`

`id, projectId, userId, workDate, hours, description, status, approvedById, approvedAt, rejectionReason`

- `status: TimesheetStatus` enum: `DRAFT, SUBMITTED, APPROVED, REJECTED`
- Validation: `0 < hours "d 24` description "d 1000 chars.

`Document`

`id, projectId, type, fileName, fileUrl, invoiceNumber, invoiceAmount, invoiceStatus, notes, uploadedById`

- `type: DocumentType` enum: `BAST, INVOICE, CONTRACT, OTHER`
- When both a BAST **and** an Invoice exist for a project, the project is auto-transitioned to `CLOSED`.

`Activity`

Lightweight feed (`type, message, projectId, userId, createdAt`) used by the "Recent Activity" widget on dashboards.

`AuditLog`

`id, userId (nullable), userName, userRole, action, entityType, entityId, description, dataBefore (Json?), dataAfter (Json?), createdAt`

- Indexed on `createdAt`, `userId`, `action`, `(entityType, entityId)`.
- `passwordHash` and other sensitive fields are stripped before storing in `dataBefore`/`dataAfter`.

---

4. Backend Architecture

4.1 Bootstrapping

`artifacts/api-server/src/index.ts` builds an Express app with: cors, cookie-parser, JSON body parser, pino-http logger, then mounts `router` from `routes/index.ts` under `/api`. Files are served from `/uploads`.

4.2 Middleware

- `requireAuth` — verifies the JWT in `Authorization: Bearer <token>` or in the `auth_token` cookie, attaches `req.user = { sub, email, role, name }`, and rejects deleted users (re-checked on every request).
- `requireRole(...roles)` — guard for role-restricted routes.

4.3 Routes

Mounted in `artifacts/api-server/src/routes/index.ts`:

File	Base path	Notes
`health.ts`	`GET /health`	Liveness check
`auth.ts`	`/auth/login`, `/auth/me`, `/auth/logout`	Login + rate limit (5/15min per IP+email), audit
`users.ts`	`/users` CRUD	Soft delete; MANAGEMENT only
`clients.ts`	`/clients` CRUD	MANAGEMENT/ADMIN_PROJECT
`projects.ts`	`/projects` CRUD + `/projects/:id/status`	Soft delete; status changes audited; auto-close on BAST+Invoice
`resources.ts`	`/projects/:id/resources`	Add/remove team
`timesheets.ts`	`/timesheets`, `/timesheets/:id/approve\`	reject` Validation, au
`documents.ts`	`/projects/:id/documents`	Upload, triggers auto-close
`uploads.ts`	`POST /uploads`	Multer disk storage at `uploads/`
`dashboard.ts`	`/dashboard/*`	Summary, profit-trend, status-breakdown, top-projects, recent-activity, pending-aging, utilization-trend, resource-utilization-detail, utilization
`capacity.ts`	`/capacity/*`	Per-week capacity grid
`audit-logs.ts`	`/audit-logs`, `/audit-logs/actions`	MANAGEMENT only; filters + pagination
`bi.ts`	`/bi/overview`	MANAGEMENT only; full BI payload

4.4 Serializers and Metrics

`src/lib/serializers.ts` exposes:

- `projectInclude` — canonical Prisma include for a project with client, sales, PM, resources (with user), and timesheets (with user).
- `computeMetrics(project)` returning `{ actualMandays, actualCost, actualProfit, marginPct, estimatedProfit }`. Cost is calculated as `:2 ††÷W'2ó,' r F –Ç•& Fp` from approved timesheets, using the per-project resource rate (falling back to `User.dailyRate`).
- `serializeProject(project)` — the API DTO.

4.5 Audit Helper

`src/lib/audit.ts` — `recordAudit(req, opts)` and `recordAuditAnon(opts)`. Sanitizes nested objects: removes `passwordHash`, converts `Date` to ISO string. Writes to `AuditLog`. Used in users, projects, timesheets, documents, resources, and auth (login + login_failed).

4.6 Login Rate Limiter

In memory `Map<key, { count, resetAt }>` keyed by `ip + ":" + email`. After 5 failures within the rolling 15 minute window, returns `HTTP 429 { error: "Too many failed login attempts. Try again in N minute(s)." }`. Successful login resets the counter for that key.

> For multi-instance deployments, replace this with a Redis-backed limiter.

4.7 Business Intelligence (`/bi/overview`)

Single MANAGEMENT-only endpoint. Query params:

- `period = month | quarter | year | custom`
- `from`, `to` — ISO dates when `period=custom`
- `principalId` — filter to one PM (matches `Project.pmId`)
- `projectType` — one of the strings in `PROJECT_TYPES` from `src/lib/projectType.ts`

Returns:

```
{
  period: { label, from, to },
  filters: { principals: [...], projectTypes: [...] },
  profitabilityByType: [{ type, revenue, cost, profit, projectCount, avgMarginPct }],
  topTypes: [{ type, avgMarginPct, profit }],     // Top 3 by avg margin
  teamPerformance: [{ principalId, principalName, principalRole,
                      revenue, cost, profit, avgMarginPct,
                      projectCount, teamSize, avgUtilizationPct }],
  forecast: [{ month, label, junior, senior, writer, admin, pm,
               totalDemandMandays, capacityMandays, shortage }], // 3 months
  forecastCapacity: { junior, senior, writer, admin, pm },
  health: {
    monthMarginPct, quarterMarginPct,
    avgProjectDurationDays,
    utilizationTrend: [{ month, label, utilizationPct, hours }],
    projectSuccessRatePct, closedProjectCount, successfulClosedCount,
    topProjects: [{ id, code, name, clientName, type,
                    revenue, cost, profit, marginPct }] // Top 5
  }
}
```

Project type is classified from `name + code + description` using regex rules in `src/lib/projectType.ts` (mirrored on the client).

Revenue per project is **prorated** to the period: if a project has `startDate` and `endDate`, only the overlapping fraction of `contractValue` is counted.

Forecast splits remaining mandays (planned " actual) of `ACTIVE` and `OBSERVATION` projects across the months between today and the project's end date; consultant demand is split 60% senior / 40% junior.

---

5. Frontend Architecture

5.1 Routing

`src/App.tsx` uses **Wouter**. All routes (except `/login`) are wrapped in `<ProtectedRoute>` which checks the auth state and redirects to `/login`.

5.2 State and Data

- **Auth** lives in `src/lib/auth.tsx` — a React context with `login`, `logout`, `user`, `token`. The token is stored in `localStorage`.
- **Server state** uses TanStack Query (`@tanstack/react-query`). Most API calls go through Orval-generated hooks in `@workspace/api-client-react`.
- **Direct calls** (when an endpoint isn't yet in OpenAPI) use `customFetch<T>(path)`, also exported from `@workspace/api-client-react`, which automatically attaches the JWT and throws `ApiError` on non-2xx.

5.3 UI Library

shadcn/ui components live under `src/components/ui/` (Button, Card, Dialog, AlertDialog, Select, Table, Tabs, Toast, etc.) — all built on Radix primitives and styled with Tailwind. The theme is dark cyber-security: background `#0F172A`, primary `#22C55E`.

5.4 Key Pages

Path	Component	Audience
`/login`	`pages/login.tsx`	All
`/`	`pages/dashboard/*` (router by role)	All
`/projects`, `/projects/new`, `/projects/:id`	`pages/projects/*`	All (writes role-gated)
`/timesheets`	`pages/timesheets/index.tsx`	All
`/approvals`	`pages/approvals/index.tsx`	PM, Management
`/resources`	`pages/resources/index.tsx`	PM, Management
`/capacity`	`pages/capacity/index.tsx`	PM, Management
`/clients`	`pages/clients/index.tsx`	Management, Admin
`/users`	`pages/users/index.tsx`	Management

`/business-intelligence`	`pages/business-intelligence/index.tsx`	Management
`/audit-logs`	`pages/audit-logs/index.tsx`	Management
`/settings`	`pages/settings/index.tsx`	All

5.5 Charts

All charts use **Recharts**. Common idioms:

- dark grid `stroke="#1f2937"`, axis `stroke="#94a3b8"`,
- tooltip styled with the page's card background,
- primary green `#22c55e` for positive, `#ef4444` for negative,
- category palette: blue / green / purple / orange / pink / amber.

---

6. Security Model

- **Passwords** hashed with `bcryptjs` (cost 10).
- **JWT** signed with `SESSION_SECRET`. Default expiration is set in `routes/auth.ts`. The token is attached as `Bearer` and rejected if the user has been soft-deleted.
- **Login rate limiting** — see §4.6.
- **Authorization** — `requireRole` enforces RBAC at the route level. The client also hides UI elements per role for UX, but the server is the source of truth.
- **Audit logging** — every write operation produces an `AuditLog` row with before/after JSON snapshots. Sensitive fields (`passwordHash`) are stripped.
- **Input validation** — zod schemas + manual checks: password "e 6 chars, daily rate "e 0, hours `0 < x "d 24`, description "d 1000 chars, money fields non-negative.
- **Soft delete** — Project and User both retain history; queries filter `deletedAt: null` by default.
- **CORS** — wide-open in dev; should be tightened for production by configuring an allowed origin list.

---

7. Build, Run, Deploy

7.1 Local development

Workflows defined in `.replit` artifact configs auto-start:

- `artifacts/api-server: API Server` — `pnpm --filter @workspace/api-server run dev`
- `artifacts/web: web` — `pnpm --filter @workspace/web run dev`
- `artifacts/mockup-sandbox: Component Preview Server` — design sandbox

The API server bundles via esbuild (`build.mjs`) and runs the bundle. Vite serves the SPA in dev.

7.2 Environment Variables

Variable	Required	Notes
`DATABASE_URL`	yes	Provided by Replit Postgres
`SESSION_SECRET`	yes	JWT signing secret
`PORT`	yes	Provided by the artifact runtime
`NODE_ENV`	no	`development` in dev workflow, `production` after deploy

7.3 Database Migrations

The schema is managed with `prisma db push` (no migration history in this template). To change the schema:

1. Edit `lib/db/prisma/schema.prisma`. 2. Run `pnpm --filter @workspace/db exec prisma db push`. 3. Run `pnpm --filter @workspace/db exec prisma generate`. 4. Restart the API server workflow.

7.4 Seeding

`lib/db/src/seed.ts` populates demo users, clients, and projects. Default password for all seeded users is `password123`. Seeded emails include:

- `management@secureprofit.id`
- `pm@secureprofit.id`
- `sales@secureprofit.id`
- `konsultan@secureprofit.id`, `konsultan2@secureprofit.id`
- `writer@secureprofit.id`
- `admin@secureprofit.id`

7.5 Deployment

The app is deployed via Replit's Autoscale deployment. The build pipeline:

1. Install dependencies (`pnpm install`). 2. `pnpm run build` (typecheck + per-artifact build). 3. The API server runs the esbuild bundle; the web artifact runs `vite preview`.

After deploy, the production app is available at the configured `.replit.app` domain or custom domain, behind Replit's TLS-terminating proxy.

---

8. Testing

End-to-end tests use the Replit Playwright testing harness (`runTest`). Verified flows:

- Login (success and failure)
- Login rate limiter (HTTP 429 after 5 failures)
- Audit log endpoint (filtering, pagination, sanitized payload)
- BI dashboard (filters, all sections rendered, role-gated)

For local sanity checks the `curl + node` script in the README of each service can be used to verify a

JWT login round-trip.

---

9. Known Limitations / Future Improvements

- Login rate limiter is in-memory — swap for Redis for multi-instance deployments.
- OpenAPI spec is not yet regenerated for the `/audit-logs` and `/bi` endpoints — they are called via `customFetch`. Run Orval to regenerate typed React Query hooks.
- No background job runner — auto-close on BAST+Invoice happens inline on upload. If async processing is needed, introduce a queue (e.g. BullMQ with Redis).
- No file scanning on uploads — files are stored on local disk under `uploads/`. For production, consider object storage (S3/Replit App Storage) and AV scanning.
- Project type classification is regex-based. To make it deterministic, add a `projectType` column to `Project` and let users pick it on creation.
- `Activity` and `AuditLog` partially overlap by design: Activity is the human-readable feed, AuditLog is the immutable forensic record with before/after snapshots.